SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: **Database Management Systems**
COURSE CODE: **CSA0504**

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr

Code of Conduct:

I R.Pranathi(192525076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: R.Pranathi

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs

Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.

Pseudocode: Query Processing Pipeline

BEGIN

1. Read SQL Query

2. Perform Lexical Analysis

   - Convert SQL query into tokens.

3. Perform Syntax Analysis

   - Check the grammar of the SQL query.

   - Create a parse tree.

4. Perform Semantic Analysis

   - Check table names and column names.

   - Check data types and permissions.

5. Generate Logical Query Plan

   - Convert the parse tree into a logical plan.

6. Optimize the Query

   - Generate different execution plans.

   - Estimate the cost of each plan.

   - Select the plan with the lowest cost.

7. Generate Physical Execution Plan

- Select suitable operations such as:

  Table Scan, Index Scan, Join, Sort, etc.

8. Execute the Query

- Execute the selected physical plan.

- Produce the final result.

9. Display Result

- Format and return the result to the user.

END

Simple Pipeline

SQL Query
↓
Lexical Analysis
↓
Parsing
↓
Semantic Analysis
↓
Logical Query Plan
↓
Query Optimization
↓
Physical Execution Plan
↓
Query Execution
↓
Result Output

2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

```
BEGIN
Input:
   Relations R1, R2, R3, ..., Rn
   Number of pages for each relation
   Join conditions
Output:
   Join ordering with minimum I/O cost
1. Generate all possible join orderings.
2. FOR each join ordering DO
      Set Total_Cost = 0
      FOR each join in the ordering DO
         Estimate the number of pages
         that must be read from both relations.
         Calculate I/O cost of the join.
         Join_Cost =
            Pages_Read_From_Left +
            Pages_Read_From_Right
         Add Join_Cost to Total_Cost.
      END FOR
      Store the join ordering and its Total_Cost.
  END FOR
3. Compare the Total_Cost of all join orderings.
4. Select the join ordering with
```

the minimum Total_Cost.

5. Return the selected join ordering

as the optimal query execution plan.

END

3. Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

PSEUDOCODE:

Algorithm Nested_Loop_Join(R, S)

Input:

R = Outer relation

S = Inner relation

BR = Number of buffer pages occupied by R

BS = Number of buffer pages occupied by S

NR = Number of tuples in R

NS = Number of tuples in S

B = Number of available buffer pages

Output:

Joined tuples

```
BEGIN
    FOR each tuple r in relation R DO
        FOR each tuple s in relation S DO
            IF Join_Condition(r, s) is TRUE THEN
                Output (r, s)
            END IF
```

```
    END FOR
  END FOR
END
```

I/O Cost Analysis

For the **basic Nested Loop Join**, if R is the outer relation:

Cost = BR + (NR × BS)

Where:

- BR = number of pages in outer relation R
- BS = number of pages in inner relation S
- NR = number of tuples in R

The outer relation is scanned once, and the inner relation is scanned once for **every tuple of R**.

Example

Suppose:

R has 100 pages
S has 200 pages
R contains 1,000 tuples

Then:

Cost = BR + (NR × BS)

= 100 + (1,000 × 200)

= 200,100 I/O operations

Therefore, the time complexity in terms of tuple comparisons is:

O(NR × NS)

and the page I/O cost is:

O(BR + NR × BS)

Conclusion: Nested Loop Join is simple to implement, but it can be expensive when the outer relation has many tuples and the inner relation occupies many pages.

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

PSEUDOCODE:

```
Algorithm Detect_Deadlock_WFG

Input:

   Set of transactions T

   Wait-For Graph WFG

Output:

   Deadlock detected or No deadlock

BEGIN

1. Create a Wait-For Graph (WFG).

2. Add each transaction as a node in the graph.

3. FOR each waiting transaction Ti DO

      IF Ti is waiting for transaction Tj THEN

         Add an edge Ti → Tj

      END IF

  END FOR

4. Perform cycle detection using DFS.

5. FOR each transaction Ti DO

      IF Ti is not visited THEN
```

```
        IF DFS(Ti) finds a cycle THEN
            Display "Deadlock Detected"
            Display the transactions involved in the cycle
            STOP
        END IF
      END IF
  END FOR
6. Display "No Deadlock Detected".
END
Algorithm DFS(Ti)
BEGIN
   Mark Ti as VISITED
   Mark Ti as CURRENT in recursion stack
   FOR each transaction Tj connected from Ti DO
      IF Tj is CURRENT THEN
         RETURN TRUE
      END IF
      IF Tj is not VISITED THEN
         IF DFS(Tj) = TRUE THEN
            RETURN TRUE
         END IF
      END IF
   END FOR
```

Remove Ti from CURRENT

RETURN FALSE

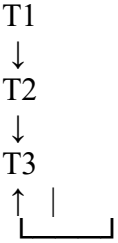
END

Example

Suppose the transactions are:

T1 → T2
T2 → T3
T3 → T1

The WFG contains a cycle:



Therefore:

Cycle exists → Deadlock Detected

If there is **no cycle**, then:

No cycle → No Deadlock

5. Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

PSEUDOCODE:

Algorithm Basic_Two_Phase_Locking(Transaction T)

BEGIN

1. Start Transaction T

2. Growing Phase

WHILE T needs to access a data item DO

Request a lock on the data item.

IF the lock is compatible with existing locks THEN

Grant the lock to T

Continue execution

ELSE

Put T into WAIT state

Wait until the required lock becomes available

Grant the lock to T

END IF

END WHILE

3. Shrinking Phase

When T finishes acquiring all required locks:

FOR each lock held by T DO

Release the lock

END FOR

4. Commit Transaction T

5. End Transaction T

END

Simple Flow

Transaction Starts
↓
Request Lock
↓
Is Lock Available?
↙ ↘
YES NO

↓ ↓
Grant Wait
Lock ↓
↓ Lock Available
Access ↓
Data Grant Lock
↓ ↓
More Operations
↓
Growing Phase Ends
↓
Release Locks
↓
Commit
↓
Transaction Ends

Important: In Basic 2PL, the transaction has two phases:

- **Growing phase:** Locks can be acquired, but not released.
- **Shrinking phase:** Locks can be released, but no new locks can be acquired.

6. Write pseudocode for timestamp-based concurrency control using Thomas Write Rule for handling conflicting read/write operations.

PESUDOCODE:

Timestamp-Based Concurrency Control Using Thomas' Write Rule

Algorithm Thomas_Write_Rule(Transaction T, Data Item X)

Input:
TS(T) = Timestamp of transaction T
Read_TS(X) = Largest timestamp of any transaction that read X
Write_TS(X) = Largest timestamp of any transaction that wrote X

BEGIN

1. When Transaction T wants to READ(X):

IF TS(T) < Write_TS(X) THEN
Abort T

```
    Display "Read rejected"
  ELSE
    Allow READ(X)
    Read_TS(X) = max(Read_TS(X), TS(T))
  END IF
```

2. When Transaction T wants to WRITE(X):

```
  IF TS(T) < Read_TS(X) THEN
    Abort T
    Display "Write rejected"

  ELSE IF TS(T) < Write_TS(X) THEN
    Ignore the WRITE operation
    Display "Obsolete write ignored"
    Continue transaction

  ELSE
    Perform WRITE(X)
    Write_TS(X) = TS(T)
  END IF
```

3. Continue execution until all operations are completed.

4. If no operation causes an abort:
   Commit Transaction T.

END

Thomas' Write Rule

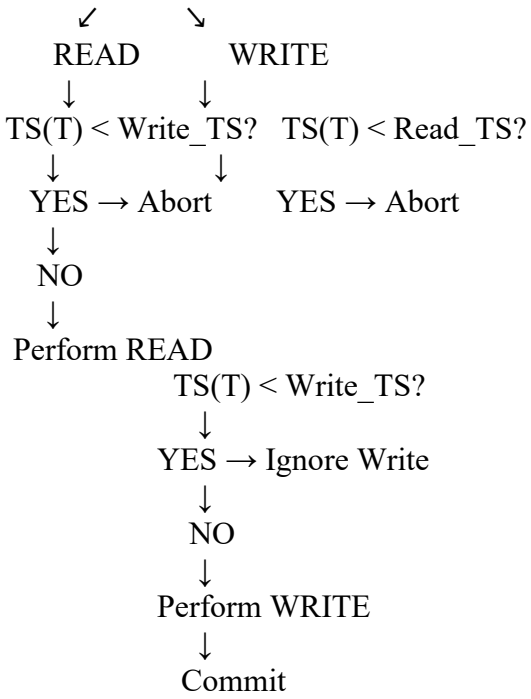
The important condition is:

```
IF TS(T) < Write_TS(X)
THEN
  Ignore the WRITE operation
```

Instead of aborting the transaction, **Thomas' Write Rule ignores an obsolete write** because a newer transaction has already written the value.

Simple Flow

```
Transaction T
     ↓
Read or Write X?
```



7. Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.

PSEUDOCODDE:

ARIES Recovery Algorithm

Algorithm ARIES_Recovery

Input:
  Log file
  Database
  Last Checkpoint

Output:
  Recovered and Consistent Database

BEGIN

// Phase 1: Analysis

1. Read the log starting from the last checkpoint.

2. Create:
    Transaction Table
    Dirty Page Table

3. Scan the log records.

4. FOR each log record DO

    IF a transaction starts THEN
      Add transaction to Transaction Table.

    ELSE IF a page is modified THEN
      Add the page to Dirty Page Table
      if it is not already present.

    ELSE IF a transaction commits THEN
      Mark transaction as COMMITTED.

    ELSE IF a transaction aborts THEN
      Mark transaction as ABORTING.

    END IF

  END FOR

5. Identify:
    - Active transactions
    - Dirty pages
    - Starting point for Redo

// Phase 2: Redo

6. Start Redo from the smallest required log sequence number.

7. FOR each update log record DO

    IF the update needs to be redone THEN
      Apply the update to the database.
    END IF

  END FOR

8. Restore all necessary changes recorded in the log.

// Phase 3: Undo

9. Identify all transactions that were active
   when the system crashed.

10. Add these transactions to the Undo list.

11. WHILE Undo list is not empty DO

    Select the transaction with the largest
    Log Sequence Number (LSN).

    Undo its last update.

    Write a compensation log record (CLR).

    Move to the previous log record
    of that transaction.

    IF all operations of the transaction
    have been undone THEN
       Write an END record.
       Remove transaction from Undo list.
    END IF

  END WHILE

12. Write all required END records.

13. Database is now recovered and consistent.

END

ARIES Recovery Flow

System Crash
  ↓
Analysis Phase
  ↓
Find active transactions
and dirty pages
  ↓
Redo Phase
  ↓
Repeat necessary logged
changes
  ↓
Undo Phase

↓
Undo incomplete transactions
using CLRs
↓
Consistent Database

In short:

- Analysis → Finds what was happening at the time of crash.
- Redo → Reapplies necessary changes.
- Undo → Removes changes made by incomplete transactions.

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

PSEUDOCODE:

Shadow Paging Recovery

Algorithm Shadow_Paging_Recovery

Input:
Database Pages
Shadow Page Table

Output:
Recovered Database

BEGIN

1. Create Shadow Page Table.

2. Copy the current page table to the Shadow Page Table.

3. Set:
Shadow_Page_Table = Current_Page_Table

4. Start Transaction.

5. WHILE transaction is running DO

IF a page needs to be modified THEN

Create a new copy of the page.

Modify the new page.

Update Current_Page_Table
to point to the new page.

Keep Shadow_Page_Table unchanged.

END IF

END WHILE

6. IF Transaction completes successfully THEN

Write all modified pages to stable storage.

Make Current_Page_Table
the new Shadow_Page_Table.

Commit Transaction.

ELSE IF System Failure occurs THEN

Discard Current_Page_Table.

Restore:
Current_Page_Table = Shadow_Page_Table

Discard all modified pages.

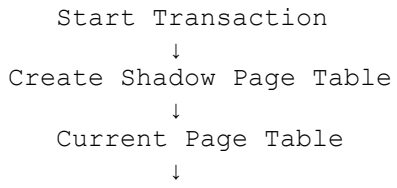
Roll back the database
to its state before the transaction.

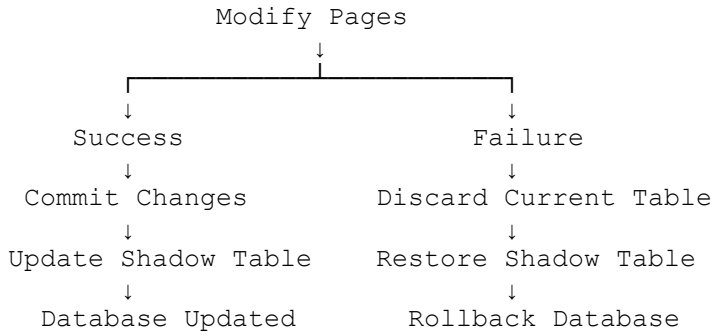
END IF

7. End Transaction.

END

Simple Flow





9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

PSEUDOCODE:

Immediate Update Recovery Using UNDO/REDO

Algorithm Immediate_Update_Recovery

Input:
Database
UNDO/REDO Log
Checkpoint

Output:
Recovered and Consistent Database

BEGIN

// Normal Transaction Processing

1. Start Transaction T.

2. Before modifying a data item X:
Write log record:
<T, X, Old_Value, New_Value>

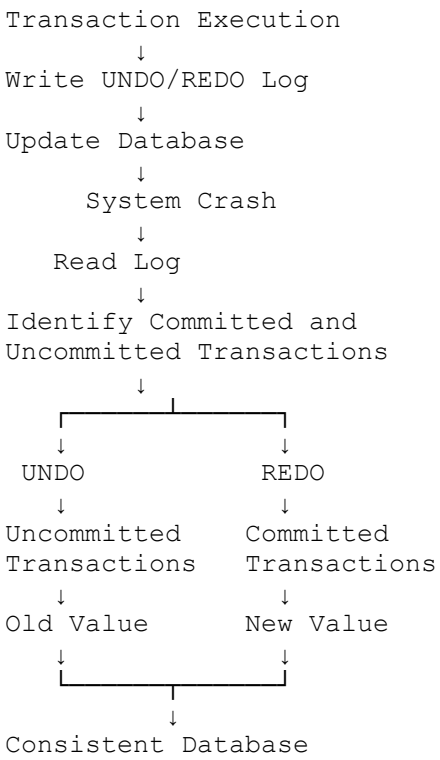
3. Write the log record to stable storage.

4. Modify X in the database.

5. IF Transaction T commits THEN
Write <T, COMMIT> to the log.
Write log to stable storage.

```
  END IF


// Recovery After System Crash

6. Read the log from the last checkpoint.

7. Identify:
      Committed transactions
      Uncommitted transactions

// UNDO Phase

8. FOR each uncommitted transaction T DO

      Scan its log records backward.

      FOR each update <T, X, Old_Value, New_Value> DO
         Restore X = Old_Value.
      END FOR

      Write <T, ABORT> to the log.

  END FOR


// REDO Phase

9. FOR each committed transaction T DO

      Scan its log records forward.

      FOR each update <T, X, Old_Value, New_Value> DO
         Set X = New_Value.
      END FOR

  END FOR

10. Write all recovery information to stable storage.

11. Database is now consistent.

END
```

Simple Flow



UNDO: Restores the **old values** of uncommitted transactions.

REDO: Reapplies the **new values** of committed transactions.

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

PSEUDOCODE:

Database Connectivity Manager

Algorithm Database_Connectivity_Manager

Input:
  Database URL
  Username
  Password
  SQL Query

Output:
  Query Result

BEGIN

1. Initialize Connection Pool.

2. Create a fixed number of database connections.

3. Store all available connections in the Connection Pool.

4. Request a connection from the pool.

5. IF a connection is available THEN
    Get the connection.
  ELSE
    Wait until a connection becomes available.
  END IF

6. Start a database transaction.

7. TRY

    Prepare the SQL Query.

    Execute the query.

    IF query is successful THEN
      Commit the transaction.
      Return the query result.
    END IF

8. CATCH Database Exception

Display the error message.

Rollback the transaction.

Cancel all changes made by the transaction.

9. FINALLY

Return the connection to the Connection Pool.

10. IF the connection is invalid THEN
Close the connection.
END IF

11. Continue using the Connection Pool
for other database requests.

END

Simple Flow